

DOCUMENTATION TECHNIQUE

PROJET 3

NetKeep

Gestion de Parc & Helpdesk

PHP orienté objet

MySQL

Architecture MVC

FORMATION

BTS SIO SLAM

DURÉE

1 mois

CONTEXTE

Gestion interne du parc et des tickets

LIVRABLES

Code · BDD · PDF

Projet 3 du BTS SIO SLAM

Durée : 1 mois

Équipe : 3 étudiants

Technologies : PHP POO, MySQL, MVC

ÉQUIPE PROJET

Arkam

Bilel

Ilyès

PK : clé primaire

FK : clé étrangère

Relations conformes au besoin

utilisateurs

auth

id : INT
nom : VARCHAR(255)
prenom : VARCHAR(255)
email : VARCHAR(100) UNIQUE
password : VARCHAR(255)
role : ENUM(employe, technicien)
niveausupport : TINYINT NULL

matériels

inventory

id : INT
nom : VARCHAR(100)
type : ENUM
numserie : VARCHAR(50) UNIQUE
dateachat : DATE
statut : ENUM(actif, enreparation, archive)
estarchive : TINYINT
idutilisateur : INT NULL

tickets

helpdesk

id : INT
titre : VARCHAR(150)
description : TEXT
urgence : ENUM(faible, moyen, critique)
typeprobleme : ENUM(materiel, logiciel, reseau, autre)
statut : ENUM(ouvert, encours, resolu)
niveauescalade : TINYINT
messageresolution : TEXT NULL
idmateriel : INT NULL
idauteur : INT
idtechnicien : INT NULL

données de test

seed

10 utilisateurs

30 matériels environ

tickets ouverts / en cours / résolus

cas réseau, logiciel, matériel

contraintes

sql

UNIQUE sur email
UNIQUE sur numserie
FK idutilisateur → utilisateurs.id
FK idauteur → utilisateurs.id
FK idtechnicien → utilisateurs.id

suppression

rules

matériel : archivage prévu

ON DELETE SET NULL sur affectations

statuts gérés par contrôleurs

cohérence logique métier

1 utilisateur → N matériels

Un employé peut avoir plusieurs équipements affectés, ou aucun.

1 utilisateur → N tickets créés

Chaque ticket conserve son auteur via idauteur.

1 ticket → 0..1 matériel

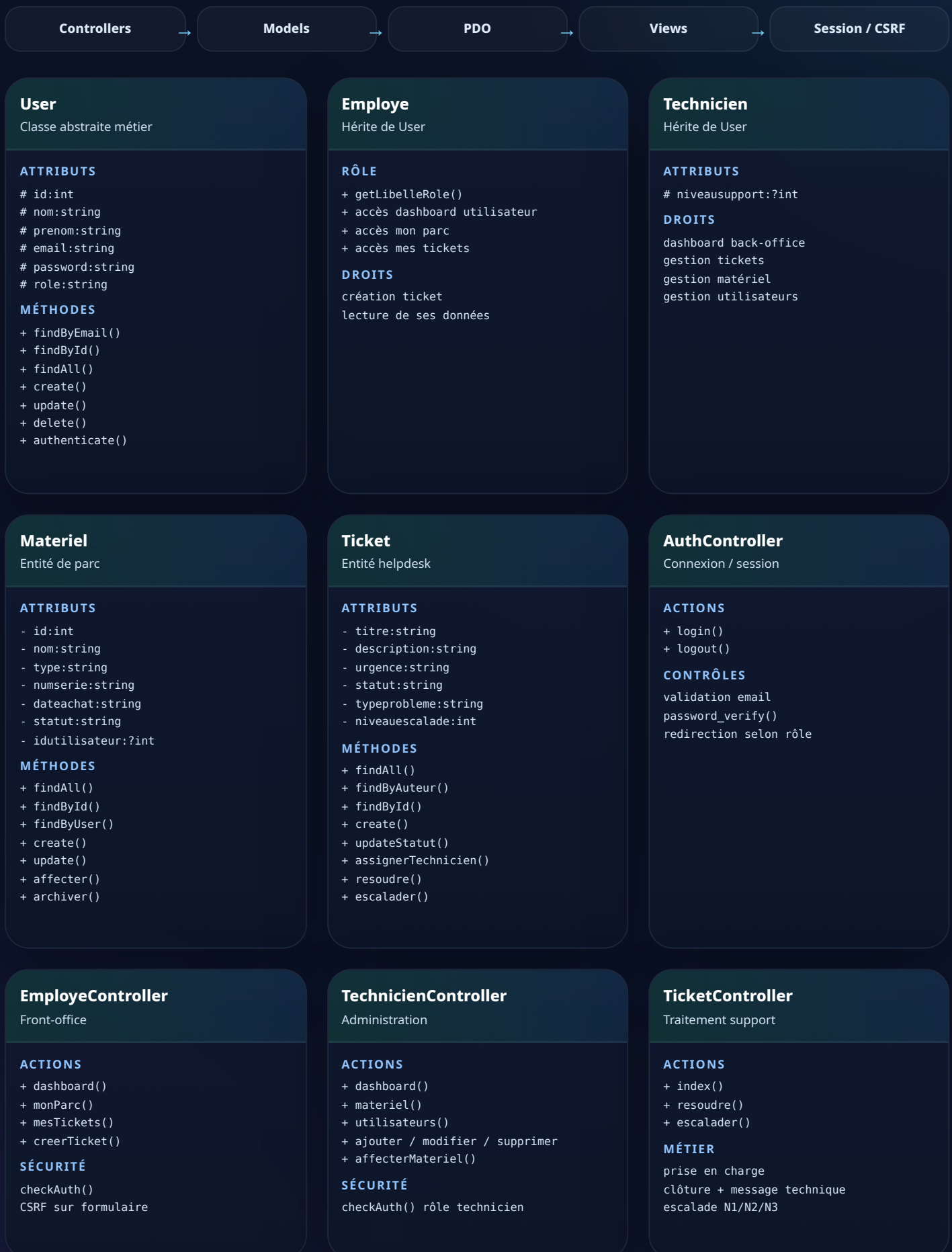
Un ticket logiciel ou réseau peut ne pas cibler un appareil précis.

1 technicien → N tickets traités

Le technicien assigné est mémorisé via idtechnicien.

Archivage métier

Le parc garde un historique via estarchive et statut.



Pré-requis

Serveur	Apache + PHP 8.x
BDD	MySQL / MariaDB
Style	Bootstrap Icons + CSS maison
Architecture	MVC en dossiers séparés

Arborescence attendue

```
/app
/controllers
/models
/views
/public/css
/config
/sql
index.php
```

Sécurité obligatoire

- Utiliser `password_hash()` et `password_verify()`.
- Passer toutes les requêtes BD par PDO préparé.
- Bloquer l'accès aux pages admin pour les employés.
- Vérifier les formulaires et les jetons CSRF.

Étapes d'installation

- 1 Créer la base bddnetkeep puis importer le fichier SQL contenant structure et données de test.
- 2 Configurer les accès MySQL dans `config/database.php` : hôte, nom de base, utilisateur et mot de passe.
- 3 Déposer le projet sur Apache, vérifier le point d'entrée `index.php` et l'accès au dossier public CSS.
- 4 Tester la connexion, puis contrôler les rôles employé / technicien et les accès protégés.

```
CREATE DATABASE bddnetkeep;
-- importer sql/netkeep.sql
-- configurer config/database.php
-- lancer Apache/MySQL
-- ouvrir /index.php
```

Jeu de données fourni

TYPE	EXEMPLE	STATUT	USAGE
Utilisateurs	employe / technicien	OK	Tests de rôles et navigation
Matériels	PC / écran / serveur	Actifs + archive	Suivi du parc et affectations
Tickets	ouvert / en cours / résolu	Multi-cas	Support, résolution, escalade

Contrôles finaux

- Un employé voit uniquement son parc et ses tickets.
- Un technicien gère le parc, les utilisateurs et la totalité des tickets.
- Les identifiants invalides déclenchent un message d'erreur lisible.
- L'archivage retire le matériel du parc actif sans perte de traçabilité.

Points techniques marquants

- Mise en place d'une architecture MVC claire pour séparer l'affichage, la logique métier et l'accès aux données.
- Utilisation de classes PHP pour gérer les entités principales : utilisateurs, matériels et tickets.
- Sécurisation des accès avec authentification, rôles différenciés et requêtes préparées.
- Gestion des opérations métier essentielles : affectation du matériel, prise en charge des tickets et archivage.
- Structuration de la base de données avec des relations cohérentes et exploitables.